

Express / express-generator — Réponses basées sur le projet **myapp**

Document généré pour le projet `~/Projects/myapp`.

1. Quels fichiers et dossiers sont créés par express-generator ?

Dans **myapp**, on retrouve la structure typique du générateur :

- **Racine** : `package.json`, `app.js`
- **bin/www** : démarrage du serveur
- **routes/** : `index.js`, `users.js`
- **views/** : `index.ejs`, `error.ejs`
- **public/** : au minimum `stylesheets/style.css`
- **node_modules/** : après `npm install`

Script de démarrage dans `package.json` :

```
"scripts": {  
  "start": "node ./bin/www"  
}
```

2. À quoi sert le fichier `app.js` ?

Dans **myapp**, `app.js` :

1. Charge les dépendances et les routeurs (`index`, `users`).
2. Crée l'application `express()`.
3. Configure les **vues** (dossier + moteur EJS).
4. Enregistre les **middlewares** (logs, JSON, formulaires, cookies, fichiers statiques).
5. **Monte** les routes sur `/` et `/users`.
6. Ajoute la gestion **404** puis le **gestionnaire d'erreur** qui rend `error`.
7. **Exporte** `app` pour `bin/www`.

3. Comment les routes sont-elles ajoutées dans `app.js` ?

Par **app.use(préfixe, router)** :

```
app.use('/', indexRouter);  
app.use('/users', usersRouter);
```

- Ce que définit `indexRouter` sur `/` répond à l'URL **/**.
- Ce que définit `usersRouter` sur `/` répond à **/users**.

4. Comment importer / exporter les routes ?

Dans `app.js` :

```
var indexRouter = require('./routes/index');
var usersRouter = require('./routes/users');
```

Dans `routes/index.js` (exemple) :

```
var express = require('express');
var router = express.Router();

router.get('/', function(req, res, next) {
  res.render('index', { title: 'Express' });
});

module.exports = router;
```

Même principe dans **routes/users.js** avec `module.exports = router`.

myapp utilise **CommonJS** (`require` / `module.exports`), pas `import` / `export`.

5. Où sont définies les routes ?

Dans le dossier **routes/** :

- **routes/index.js** → page d'accueil (GET `/`).
- **routes/users.js** → listing utilisateurs (GET `/users`).

6. À quoi sert le dossier `routes/` ?

Il évite de tout mettre dans `app.js`. `app.js` fait `require` + `app.use(...)` ; les chemins et handlers sont dans des fichiers séparés, plus faciles à faire évoluer.

7. Comment définit-on une route en Express ? (dans myapp)

Sur un **Router** avec une méthode HTTP + un chemin + une fonction.

Accueil (`routes/index.js`) :

```
router.get('/', function(req, res, next) {
  res.render('index', { title: 'Express' });
});
```

Utilisateurs (`routes/users.js`) :

```
router.get('/', function(req, res, next) {  
  res.send('respond with a resource');  
});
```

Dans **myapp**, seules des routes **GET** existent dans `routes/`.

8. Comment une route traite une requête sur une URL ?

Express associe **méthode + chemin** (avec le préfixe de `app.use`).

- **GET /**** → `res.render('index', { title: 'Express' })` → HTML depuis `views/index.ejs`.
- **GET /users** → `res.send('respond with a resource')`.

En fin de `app.js`, un middleware sans chemin gère le **404** puis le **gestionnaire d'erreur** appelle `res.render('error')`.

9. Paramètres `req`, `res`, `next`

Handlers : `function(req, res, next)`.

- **req** : requête (query, params, body si parsé, etc.).
- **res** : réponse (`res.render`, `res.send`, etc.).
- **next** : passer au middleware suivant ; ex. `next(createError(404))` pour les erreurs.

10. `.get()`, `.post()`, `.put()`, `.delete()`, etc.

myapp n'utilise que `.get()` dans `routes/`.

En pratique : **GET** lecture ; **POST** création / action ; **PUT**/**PATCH** mise à jour ; **DELETE** suppression. Tu ajouterais par ex. `router.post('/users', ...)` quand nécessaire.

11. À quoi sert `.all()` ?

myapp n'utilise pas `router.all()` ni `app.all()`. `.all(chemin, handler)` enregistre le handler pour **tous les verbes** HTTP sur ce chemin.

12. Views avec EJS dans myapp

Configuration (`app.js`) :

```
app.set('views', path.join(__dirname, 'views'));  
app.set('view engine', 'ejs');
```

****Rendu**** : ``res.render('index', { title: 'Express' })`` → ``views/index.ejs`` avec la variable ``title``.

****`views/index.ejs`**** : ``<%= title %>`` injecte la valeur ; lien CSS ``/stylesheets/style.css``.

****`views/error.ejs`**** : utilise ``message``, ``error.status``, ``error.stack`` (remplis via ``res.locals`` dans le gestionnaire d'erreur).

13. Dossier ``public`` (fichiers statiques)

```
app.use(express.static(path.join(__dirname, 'public')));
```

Les fichiers sous ****`public/`**** sont servis à la racine URL : ex. ``public/stylesheets/style.css`` → ****`/stylesheets/style.css`**** (comme dans ``index.ejs``).

14. Lien avec ``bin/www``

Le serveur HTTP démarre dans ****`bin/www`**** : ``require('..../app')``, port par défaut ****3000**** (``process.env.PORT || '3000'``), ``http.createServer(app)`` puis ``server.listen(port)``.

Chemin du projet : ``/Users/djibrilh/Projects/myapp``